

LINUX FOR DEVOPS

LINUX NETWORKING FUNDAMENTALS

A Complete Guide for DevOps Engineers

Created By
Suryakant Kulkarni



1

WHAT IS NETWORKING IN LINUX?



• DEFINITION

- Networking in Linux is the ability of a Linux system to communicate with other systems, devices, and services over a local or wide-area network.
- Linux treats nearly everything as a file - including network sockets - making networking deeply integrated into the operating system kernel itself.

• WHY NETWORKING MATTERS FOR DEVOPS

- **Server Configuration:** Every cloud server needs a correctly configured IP address, subnet, and gateway before any application can be deployed.
- **Application Deployment:** Microservices communicate over networks. A broken network config means a broken deployment pipeline.
- **Firewall & Security:** Controlling which ports are open and which IPs can connect is a core responsibility of every DevOps engineer.
- **Monitoring & Observability:** Tools like Prometheus, Grafana, and ELK all depend on network access between services.
- **Container Networking:** Docker and Kubernetes create virtual networks between containers; understanding Linux networking helps you debug them.
- **CI/CD Pipelines:** Jenkins, GitHub Actions, and GitLab Runners all require outbound network access to clone repos, push images, and deploy code.

• TYPES OF NETWORKS

Network Type	Range	Speed	Real-World Example
PAN (Personal Area Network)	Up to 10 metres	Up to 480 Mbps	Developer laptop, USB/Bluetooth
LAN (Local Area Network)	Up to 2 km	100 Mbps - 10+ Gbps	Office network, Data centre rack
MAN (Metropolitan Area Network)	5 km - 50 km	High Speed Fibre	City-wide ISP connecting branches
WAN (Wide Area Network)	Beyond 50 km	Variable	The Internet, AWS global backbone

• NETWORK LAYERS

- Network layers are conceptual levels that organize how devices communicate over a network. These layers divide networking tasks into smaller parts so that communication becomes structured and easier to manage.

• OSI MODEL (OPEN SYSTEMS INTERCONNECTION)

- The OSI Model is a conceptual framework used to understand how data travels across a network.
- It divides network communication into 7 different layers, where each layer performs a specific function to ensure smooth and reliable data exchange between devices.
- As a DevOps engineer, you will primarily work with Layer 3 (IP routing), Layer 4 (TCP/UDP ports), and Layer 7 (HTTP, SSH, DNS).

- **Application Layer (Layer 7):** Provides network services directly to end-user applications for communication; protocols include HTTP, FTP, SMTP, DNS.
- **Presentation Layer (Layer 6):** Translates, encrypts, and compresses data to ensure proper format and security; protocols include SSL/TLS, JPEG, MPEG.
- **Session Layer (Layer 5):** Manages and maintains sessions between applications, including setup, synchronization, and termination; protocols include NetBIOS, RPC.
- **Transport Layer (Layer 4):** Ensures reliable or fast data delivery using segmentation, flow control, and error handling; protocols include TCP, UDP.
- **Network Layer (Layer 3):** Handles logical addressing and routing to deliver data across different networks; protocols include IP, ICMP.
- **Data Link Layer (Layer 2):** Provides node-to-node data transfer with error detection using MAC addresses; protocols include Ethernet (802.3), PPP.
- **Physical Layer (Layer 1):** Transmits raw bits over physical media like cables and hardware devices; includes hubs, cables, and electrical signals.

Application Layer

Presentation Layer

Session Layer

Transport Layer

Network Layer

Data Link Layer

Physical Layer

• TCP/IP MODEL (TRANSMISSION CONTROL PROTOCOL / INTERNET PROTOCOL)

- The TCP/IP model is a four-layer conceptual framework that standardizes how data is transmitted over networks like the Internet.
- It ensures reliable communication by breaking data into packets, routing them, and reassembling them at the destination.

- **Application Layer (Layer 4):** Interacts with software applications, managing data formatting and encryption (e.g. HTTP, FTP, SMTP, and SSH)
- **Transport Layer (Layer 3):** Handles host-to-host communication, managing data flow, error detection (e.g. TCP, UDP)
- **Internet Layer (Layer 2):** Responsible for logical addressing (IP addresses) and routing packets across networks. (e.g. IP, ARP, and ICMP)
- **Network Access Layer (Layer 1):** Manages the physical transmission of data, dealing with MAC addresses, network interfaces, and hardware like Ethernet or Wi-Fi.

Application Layer

Transport Layer

Internet Layer

Network Access Layer

• KEY NETWORKING CONCEPTS

Concept	What It Means	DevOps Example
IP Address	Unique numeric label identifying a device	10.0.1.5 -- private EC2 instance
Subnet Mask	Defines network vs host portion of an IP	255.255.255.0 = /24 subnet
Default Gateway	Router that handles traffic outside local network	192.168.1.1 -- your router
DNS	Translates domain names to IP addresses	api.myapp.com → 54.200.1.10
Port	Identifies a specific service on a host	Port 443 = HTTPS, 22 = SSH
Protocol	Rules for how data is transmitted	TCP (reliable), UDP (fast)
Firewall	Controls which traffic is allowed in/out	Allow 443, block 23
Network Interface	Hardware/virtual link to the network	eth0, docker0, ens3
Routing	Deciding the path a packet takes	ip route show
Socket	Combination of IP + port identifying an endpoint	10.0.1.5:8080

2

NETWORK INTERFACES & IP ADDRESSING



• WHAT IS A NETWORK INTERFACE?

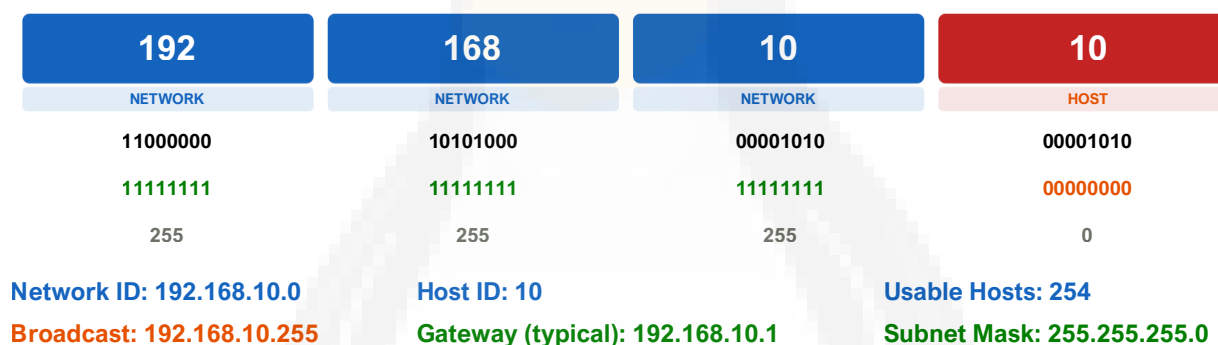
- A Network interface is the connection point between your Linux system and a network. It can be physical hardware (a NIC - Network Interface Card) or a virtual software component created by the OS. Each interface gets its own IP address and handles all traffic for that connection.
- **Physical Interface (eth0, ens3, enp0s3):** Represents a real Ethernet port on the server. Most common in bare-metal and VM environments. Name reflects the PCI bus location on modern systems.
- **Loopback Interface (lo):** A virtual interface always present at 127.0.0.1. Used by processes on the same machine to communicate with each other without going through the network.
- **Wireless Interface (wlan0, wlp2s0):** Wi-Fi adapters. Rarely used in server/DevOps environments but common on developer laptops.
- **Virtual Interface (docker0, veth0, br0, tun0):** Created by Docker, Kubernetes, or VPN software. docker0 is the bridge that lets containers talk to each other and the host.
- **VLAN Sub-Interface (eth0.10):** Logical sub-division of a physical interface. Allows one NIC to carry traffic for multiple isolated networks simultaneously.

Interface Name	Type	Description
eth0, eth1	Physical Ethernet	Legacy names - older kernels and cloud VMs
ens3, ens5	Physical Ethernet	Slot 3 or 5 on the motherboard (predictable naming)
enp0s3	Physical Ethernet	PCI bus 0, slot 3 - default in VirtualBox
eno1	Physical Ethernet	Onboard/embedded NIC named by firmware
wlan0, wlp2s0	Wireless	Wi-Fi adapters
lo	Loopback	Always 127.0.0.1 - internal process comms
docker0	Virtual Bridge	Created by Docker daemon on install
veth0, vethxxx	Virtual Ethernet	Pair used by containers and network namespaces
tun0, tap0	VPN Tunnel	Virtual tunnels used by OpenVPN, WireGuard
eth0.10	VLAN Sub-interface	VLAN ID 10 on top of eth0

• WHAT IS AN IP ADDRESS?

- An IP (Internet Protocol) address is a unique numeric label assigned to every device on a network. Think of it as a postal address for your computer - without it, data would have no idea where to go.
- IPv4 addresses are 32-bit numbers written as four decimal numbers (octets) separated by dots: 192.168.1.10. They support approximately 4.3 billion unique addresses.
- IPv6 addresses are 128-bit numbers written in hexadecimal groups: 2001:db8::1. They support 3.4×10^{38} unique addresses.
- Addresses can be static (manually set, never changes - used for servers) or dynamic (assigned by DHCP, changes periodically - used for most clients).

IPv4 ADDRESS BREAKDOWN - 192.168.10.10/24



• IPV4 VS IPV6

Feature	IPv4	IPv6
Address Length	32 bits	128 bits
Address Format	Decimal notation - 192.168.1.1	Hexadecimal notation - 2001:db8::1
Total Addresses	Approximately 4.3 billion (2^{32})	Approximately 3.4×10^{38} (2^{128})
Address Notation	Four octets separated by dots	Eight groups of four hexadecimal digits
Loopback Address	127.0.0.1	::1
Subnet Notation	192.168.1.0/24	2001:db8::/32
Header Size	20 to 60 bytes - variable length	Fixed 40 bytes - simpler processing
Configuration	Configured manually or via DHCP	Supports Manual, DHCPv6, and SLAAC
DNS Record Type	A Record	AAAA Record
Current Status	Still dominant in production environments	Adoption growing in cloud and modern infra

• TYPES OF IP ADDRESS

Type	Assigned By	Changes?	Use Case
Public IP	ISP	Dynamic or Static	Direct Internet access, web servers
Private IP	Network admin / DHCP	Usually Dynamic	Internal communication, not internet-routable
Static IP	Manually set	Never	Servers, Databases, Printers
Dynamic IP	DHCP server	Yes (lease expires)	Workstations, Developer Machines
Loopback IP	OS (127.0.0.1)	Never	Internal process communication

• IP ADDRESS CLASSES

- The 32-bit IP address is divided into five sub-classes: A, B, C, D, and E.
- Each IP address class has a specific valid range of addresses.
- The class of an IP address is determined by the pattern of bits in the first octet. This class defines how many bits are allocated to the network ID and host ID, which in turn determines the total number of networks and hosts supported.

• IP ADDRESS CLASSES - REFERENCE TABLE

Class	First Octet Range	First Bits	Default Mask	CIDR	Networks	Hosts	Use
Class A	1-126	0xxxxxxx	255.0.0.0	/8	126	16,777,214	Large enterprises, ISPs, Governments
Class B	128-191	10xxxxxx	255.255.0.0	/16	16,384	65,534	Universities, mid-size organisations
Class C	192-223	110xxxxx	255.255.255.0	/24	2,097,152	254	Home networks, small offices
Class D	224-239	1110xxxx	N/A	N/A	N/A	N/A	Multicast only
Class E	240-255	1111xxxx	N/A	N/A	N/A	N/A	Research & experimental

• PRIVATE IP ADDRESS RANGES

Class	Range	CIDR	Usable Hosts	Common Use
Class A	10.0.0.0 – 10.255.255.255	/8	16,777,214	Large enterprise networks, AWS VPCs
Class B	172.16.0.0 – 172.31.255.255	/12	1,048,574	Medium networks, Docker default bridge
Class C	192.168.0.0 – 192.168.255.255	/16	65,534	Home networks, small office LANs

• PUBLIC IP ADDRESS RANGES

Class	Range	CIDR	Usable Hosts	Common Use
Class A	1.0.0.0 – 9.255.255.255	/8	16,777,214	Large ISPs, global enterprises, cloud providers
	11.0.0.0 – 126.255.255.255			
Class B	128.0.0.0 – 172.15.255.255	/16	65,534	Universities, regional ISPs, medium enterprises
	172.32.0.0 – 191.255.255.255			
Class C	192.0.0.0 – 192.167.255.255	/24	254	Small businesses, web hosting, small ISPs
	192.169.0.0 – 223.255.255.255			
Class D	224.0.0.0 – 239.255.255.255	N/A	N/A	Multicast - streaming, OSPF, EIGRP, video conferencing
Class E	240.0.0.0 – 255.255.255.254	N/A	N/A	Reserved for research and experimental use only

• SPECIAL RESERVED ADDRESSES

Address	Purpose
0.0.0.0 – 0.255.255.255	Represents current network - non routable
127.0.0.0 – 127.255.255.255	Loopback Addresses - always refers to local machine
169.254.0.0 – 169.254.255.255	Link-local Addresses - auto assigned when DHCP fails
224.0.0.0 – 239.255.255.255	Class D - Multicast only, not assignable
240.0.0.0 – 255.255.255.254	Class E - Research and experimental only
255.255.255.255	Global broadcast address

- Why Class A has two rows: 10.0.0.0/8 is reserved for private use, so public Class A skips it and splits into two ranges.
- Why Class B has two rows: 172.16.0.0/12 is reserved for private use, so public Class B skips it and splits into two ranges.
- Why Class C has two rows: 192.168.0.0/16 is reserved for private use, so public Class C skips it and splits into two ranges.

3

SUBNETTING & CIDR NOTATION

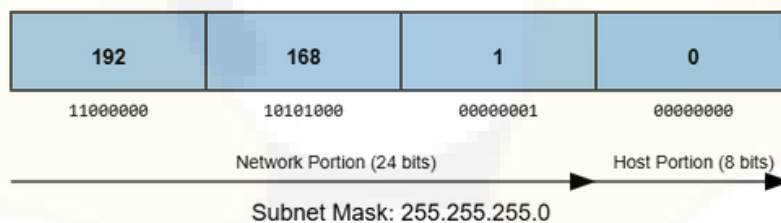


WHAT IS SUBNETTING?

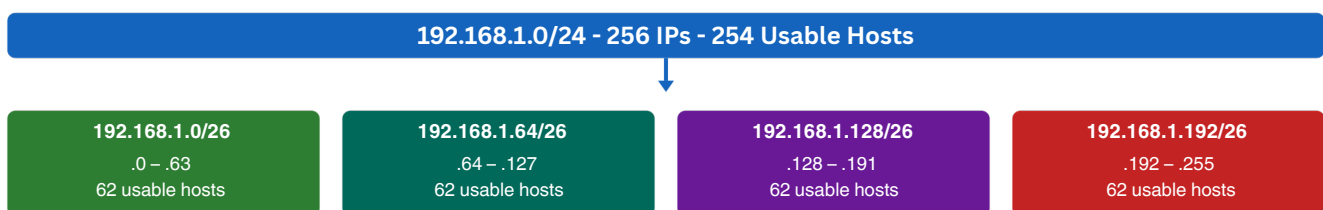
- Subnetting divides one large IP network into smaller, isolated sub-networks called subnets.
- This improves security (traffic between subnets requires routing through a firewall), reduces broadcast traffic (broadcasts stay within a subnet), and allows you to segment application tiers - for example: web subnet, app subnet, database subnet.

CIDR NOTATION EXPLAINED

- CIDR (Classless Inter-Domain Routing) is the standard way to write an IP address and its subnet mask in one short notation: 192.168.1.0/24
- The number after the slash (called the prefix length) tells you how many bits are used for the network part. The remaining bits identify individual hosts.
- **Formula:** Usable Hosts = $2^{(32 - \text{prefix})} - 2$. We subtract 2 because the first address is the network address and the last is the broadcast address - neither can be assigned to a host.
- Example: /24 gives $2^{(32-24)} - 2 = 2^8 - 2 = 256 - 2 = 254$ usable hosts
- Example: /26 gives $2^{(32-26)} - 2 = 2^6 - 2 = 64 - 2 = 62$ usable hosts
- Example: /30 gives $2^{(32-30)} - 2 = 2^2 - 2 = 4 - 2 = 2$ usable hosts (router links)



SUBNETTING - DIVIDING A /24 INTO FOUR /26 SUBNETS



Formula: Usable Hosts = $2^{(32 - \text{prefix})} - 2 \rightarrow /26: 2^{(32-26)} - 2 = 64 - 2 = 62$ hosts

• BREAKING DOWN A SUBNET - WORKED EXAMPLE

>> **Example:** 192.168.1.0/24

- **Given network:** 192.168.1.0/24
- **Subnet Mask (Decimal):** 255.255.255.0 (24 bits for network, 8 bits for hosts)
- **Subnet Mask (Binary):** 11111111 . 11111111 . 11111111 . 00000000
- **Network Address:** 192.168.1.0 (first address - cannot be assigned to a host)
- **Usable Host Range:** 192.168.1.1 to 192.168.1.254
- **Broadcast Address:** 192.168.1.255 (last address - cannot be assigned to a host)
- **Total Usable Hosts:** $2^8 - 2 = 254$
- **Default Gateway** is typically assigned the first usable host: 192.168.1.1
- **IP Class:** Class C (192.x range)

• CIDR REFERENCE TABLE

CIDR	Subnet Mask	Total IPs	Usable Hosts	Common Use
/32	255.255.255.255	1	1	Single host, loopback, route entry
/31	255.255.255.254	2	2	Point-to-point links (WAN)
/30	255.255.255.252	4	2	Router-to-router links
/29	255.255.255.248	8	6	Small network
/28	255.255.255.240	16	14	Small office / lab
/27	255.255.255.224	32	30	Department subnet
/26	255.255.255.192	64	62	Medium subnet
/25	255.255.255.128	128	126	Split network
/24	255.255.255.0	256	254	Standard LAN
/23	255.255.254.0	512	510	Large LAN
/22	255.255.252.0	1024	1022	Campus network
/20	255.255.240.0	4096	4094	Data center
/16	255.255.0.0	65536	65534	Enterprise / AWS VPC
/8	255.0.0.0	16777216	16777214	Large org
/1	128.0.0.0	2147483648	2147483646	Half internet
/0	0.0.0.0	4294967296	4294967294	Default route

4

ROUTING IN LINUX

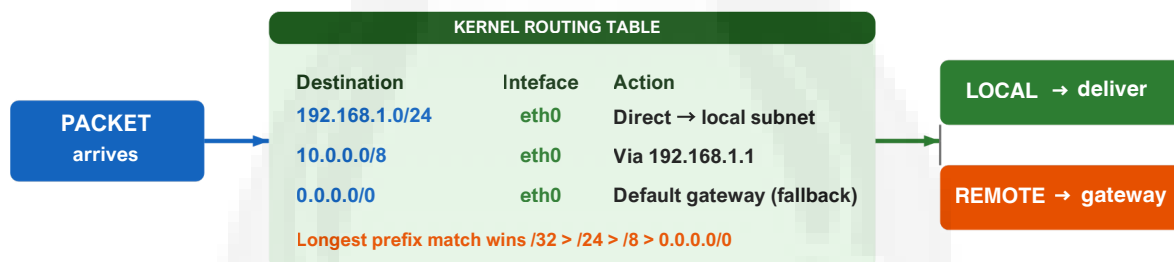


• WHAT IS ROUTING?

- Routing is the process by which the Linux kernel decides where to forward each network packet based on its destination IP address. Every packet that leaves your system goes through the kernel routing table. If the destination is on the local subnet, it is delivered directly. If not, it is sent to the default gateway (your router).

- The **Routing table** is a list of rules stored in the Linux kernel. Each rule (called a route) says: "for packets going to network X, send them out interface Y, via gateway Z."
- The kernel uses **longest-prefix matching** to pick the best route. A /32 (exact host match) beats a /24 beats a /8 beats the default route (0.0.0.0/0).

HOW LINUX ROUTES A PACKET



• TYPES OF ROUTES IN NETWORKING

- The three main types of Routes in networking are static (manually configured), dynamic (automatically learned), and default (catch-all for unknown routes).
- **Static Routes:** Manually configured by a network administrator to specify a fixed path for traffic. They are secure and efficient for small networks but hard to scale, as they require manual updates when network topology changes.
- **Dynamic Routes:** Discovered and updated automatically by routing protocols like OSPF, EIGRP, or BGP. Routers use these protocols to share network knowledge, making them ideal for large, complex networks.
- **Default Routes:** A special type of static route used when the routing table does not have a specific path to a destination. It is often used to send traffic from a stub network to an ISP (often denoted as 0.0.0.0/0).
- **Connected Routes:** Routes added automatically when an interface is activated and assigned an IP address, representing networks directly attached to the router.

5

DNS & HOSTNAME RESOLUTION



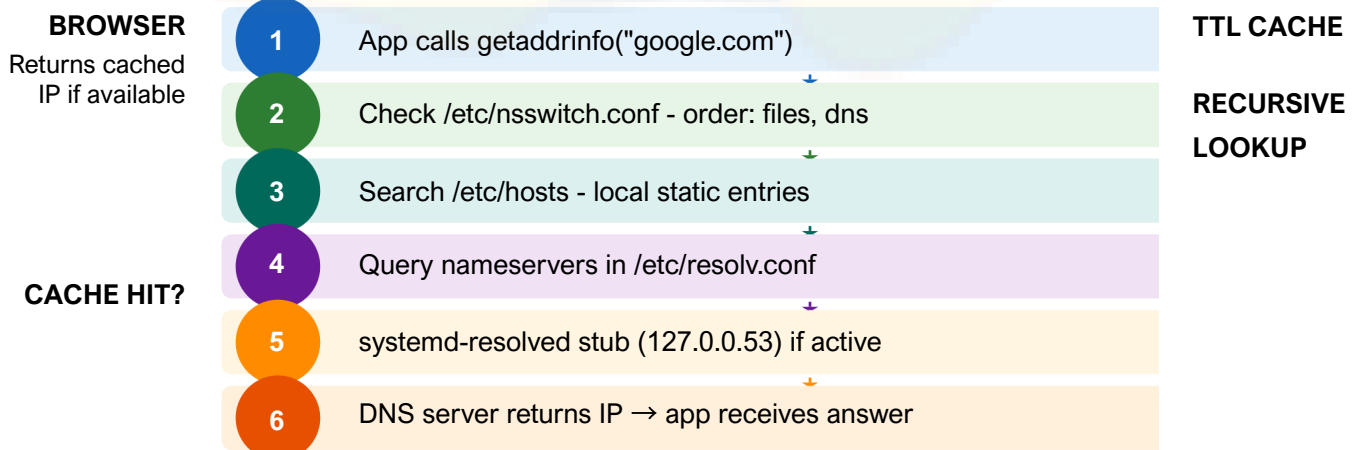
• WHAT IS DNS?

- DNS (Domain Name System) translates human-readable domain names like google.com into machine-readable IP addresses like 142.250.190.46. Without DNS, you would need to remember the IP address of every website or service you want to reach. Every Linux system queries DNS resolvers before making any network connection.

• DNS RECORD TYPES

Record Type	Full Name	Purpose	Example
A	Address	Maps hostname to IPv4 address	api.myapp.com → 54.200.1.10
AAAA	IPv6 Address	Maps hostname to IPv6 address	api.myapp.com → 2001:db8::1
CNAME	Canonical Name	Alias pointing to another hostname	www → myapp.com
MX	Mail Exchange	Specifies mail server for a domain	gmail.com → smtp.google.com
PTR	Pointer	Reverse DNS - IP to hostname	54.200.1.10 → api.myapp.com
TXT	Text	Arbitrary text (SPF, DKIM, verification)	v=spf1 include:google.com
NS	Name Server	Identifies authoritative DNS servers	example.com NS ns1.example.com
SOA	Start of Authority	Administrative info for the DNS zone	Primary NS, serial, TTL

DNS RESOLUTION FLOW IN LINUX



• KEY DNS CONFIGURATION FILES IN LINUX

File	Purpose	Example Content
<code>/etc/hostname</code>	Static hostname - read at boot	webserver-01
<code>/etc/hosts</code>	Local name-to-IP overrides - checked first	192.168.1.10 myserver
<code>/etc/resolv.conf</code>	Lists DNS server IPs and search domains	nameserver 8.8.8.8
<code>/etc/nsswitch.conf</code>	Controls resolution order (files, dns)	hosts: files dns
<code>/etc/systemd/resolved.conf</code>	Config for systemd-resolved stub resolver	DNS=8.8.8.8 8.8.4.4

• HOSTNAMES & FQDN

- A **Hostname** is the human-readable name of your Linux machine on the network - for example, web-server-01. It is stored in `/etc/hostname`.
- An **FQDN (Fully Qualified Domain Name)** combines the hostname with the domain: web-server-01.example.com. This is what DNS uses for full resolution.
- On systemd-based systems, hostnames are managed with `hostnamectl`. This tool can set three types: static (persistent), transient (runtime), and pretty (display name with spaces).
- Use `dig +trace` to watch DNS resolution step by step from the root servers down to the authoritative nameserver for any domain - very useful for debugging DNS issues.

6

PORTS & PROTOCOLS



WHAT IS A PORT?

- A port is a 16-bit number (0–65535) used by the operating system to route incoming and outgoing network traffic to the correct process.
- If an IP address is like a building's street address, then a port number is like a specific apartment number inside that building. The IP gets the packet to the right server; the port number tells the OS which specific program (service) should receive it.

PORT NUMBER RANGES (0 – 65535)

Registered Ports
1024 – 49151
App services

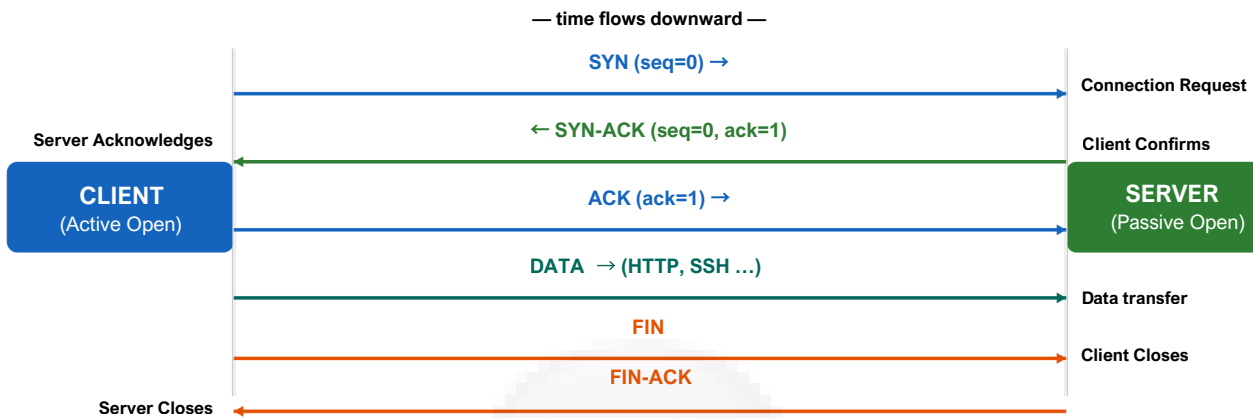
Ephemeral Ports
49152 – 65535
Client connections

Port Range	Name	Who Uses It	Example
0-1023	Well-Known Ports	System services - requires root to bind	HTTP:80, HTTPS:443, SSH:22, DNS:53
1024-49151	Registered Ports	Applications (registered with IANA)	MySQL:3306, Redis:6379, Grafana:3000
49152-65535	Ephemeral Ports	Client-side temporary connections	Your browser uses a random port here

TRANSMISSION CONTROL PROTOCOL (TCP)

- TCP provides reliable, ordered, and error-checked delivery of data between applications. It guarantees that every byte sent arrives intact and in the correct order.
- Before any data is sent, TCP performs a **Three-way Handshake** (SYN → SYN-ACK → ACK) to establish a connection. This adds a small delay but ensures both sides are ready.
- TCP uses sequence numbers and acknowledgements to track which segments have been received. If a segment is lost, the sender automatically retransmits it.
- TCP is used for: HTTP/HTTPS (web), SSH (remote access), MySQL/PostgreSQL (databases), and any application where data accuracy matters more than speed.
- The trade-off: TCP is slower due to connection setup and acknowledgement overhead. For high-speed, low-latency needs, use UDP.

TCP THREE-WAY HANDSHAKE + CONNECTION TERMINATION



• USER DATAGRAM PROTOCOL (UDP)

- UDP is a connectionless, fire-and-forget protocol. Packets (datagrams) are sent without establishing a connection first and without any acknowledgement.
- UDP is much faster than TCP because it has almost no overhead - its header is only 8 bytes vs TCP's minimum 20 bytes.
- The downside: no delivery guarantee. Packets can arrive out of order, be duplicated, or be lost entirely. The application must handle this if needed.
- UDP is used for: DNS queries (fast single request/response), live video/audio streaming (a lost frame is better than buffering), online gaming (low latency critical), and VoIP.
- Modern protocols like QUIC (used by HTTP/3) are built on UDP and add their own reliability mechanisms on top for the best of both worlds.

• INTERNET CONTROL MESSAGE PROTOCOL (ICMP)

- ICMP is a network layer protocol used for error reporting and diagnostic functions in IP networks. It does not carry application data like TCP or UDP.
- It helps devices communicate issues such as unreachable hosts, network congestion, or time exceeded during packet transmission.
- ICMP is commonly used by tools like ping (Echo Request/Reply) and traceroute to test connectivity and measure network paths.
- ICMP does not guarantee delivery and is typically used for control and monitoring, not regular data communication.

• COMMON PORTS EVERY DEVOPS ENGINEER MUST KNOW

Port	Protocol	Service	DevOps Purpose
21	TCP	FTP	File transfer (insecure) - avoid in production
22	TCP	SSH	Secure remote server access - your primary admin tool
23	TCP	TELNET	Remote access (insecure) - deprecated, avoid
25	TCP	SMTP	Email sending (server-to-server mail transfer)
53	TCP/UDP	DNS	Name resolution - must be open for all servers
67/68	UDP	DHCP	Dynamic IP assignment - used by cloud metadata agents
80	TCP	HTTP	Unencrypted web traffic - redirect to HTTPS in production
443	TCP	HTTPS	Encrypted web traffic - REST APIs, web apps
465	TCP	SMTPS	Secure email transmission (deprecated but used)
587	TCP	SMTP (Submission)	Secure email sending (modern standard)
990	TCP	FTPS	Secure FTP over TLS
2375	TCP	Docker API	Docker daemon (unencrypted) - NEVER expose publicly
2376	TCP	Docker TLS	Docker daemon (TLS encrypted) - remote Docker management
2379	TCP	etcd	Kubernetes key-value store - critical for cluster state
3000	TCP	Grafana	Grafana dashboards - monitoring UI
3306	TCP	MySQL	MySQL database - lock down with firewall rules
5432	TCP	PostgreSQL	PostgreSQL database - restrict to app subnet only
5601	TCP	Kibana	ELK Stack dashboard - Elasticsearch visualization
6379	TCP	Redis	Redis cache/queue - also lock down to app subnet
6443	TCP	K8S API Server	Kubernetes control plane - must be highly secured
8080	TCP	HTTP Alt	Jenkins, Tomcat, app proxies - common dev port
9090	TCP	Prometheus	Metrics collection endpoint - scrapes from exporters
9200	TCP	Elasticsearch	Elasticsearch REST API - log storage and search
27017	TCP	MongoDB	NoSQL database - restrict access
30000 – 32767	TCP	NodePort (K8s)	Exposes services externally in Kubernetes

7

NETWORK SERVICES & DAEMONS



• WHAT ARE NETWORK DAEMONS?

- Network daemons are background processes that start at boot and continuously manage your network configuration. They handle tasks like obtaining an IP from DHCP, maintaining DNS settings, and reacting when a network cable is plugged in or removed. Misconfigured daemons are one of the most common causes of connectivity failures in production.

• NETWORK MANAGEMENT DAEMONS (LINUX)

1. Network Manager

- Network Manager is the default network management service on RHEL, CentOS, Fedora, and Ubuntu Desktop.
- It manages different types of network connections such as Ethernet, Wi-Fi, VPN, and mobile broadband.
- Its configuration files are stored in the `/etc/NetworkManager/` directory.
- It can be managed using `nmcli` (command-line interface) or `nmtui` (text-based interface) and dynamically reacts to hardware changes.

2. systemd-networkd

- `systemd-networkd` is a lightweight network management daemon designed mainly for servers and containerized environments.
- It is configured using `.network` files located in the `/etc/systemd/network/` directory.
- It integrates closely with the `systemd` ecosystem to provide fast and efficient network configuration.
- It is commonly used in minimal and container-focused operating systems such as CoreOS.

3. systemd-resolved

- `systemd-resolved` is a system service that provides DNS resolution with local caching capabilities.
- It uses a local stub resolver that listens on the IP address `127.0.0.53`.
- Its configuration file is located at `/etc/systemd/resolved.conf`.
- It is managed using the `resolvectl` command and improves DNS lookup performance by caching results.

• DYNAMIC HOST CONFIGURATION PROTOCOL (DHCP)

- DHCP automatically assigns an IP address, subnet mask, default gateway, and DNS server to any device that connects to the network. Without DHCP, every device would need manual static configuration.
- The DHCP process follows the DORA sequence: Discover → Offer → Request → Acknowledge. This four-step handshake completes in milliseconds and gives the client a temporary IP lease.
- DHCP leases are time-limited. A typical home router gives 24-hour leases. Cloud providers often give shorter leases (a few hours). The client must renew before expiry. Lease duration depends on network configuration (cloud or on-premises).
- On Linux, DHCP is handled by NetworkManager, systemd-networkd, or the standalone dhclient daemon. Lease information is saved in `/var/lib/dhcp/dhclient.leases`.
- In cloud environments (AWS/GCP/Azure), DHCP assigns private IPs and the instance metadata service provides additional config. Never disable DHCP on a cloud instance.

DHCP - DYNAMIC IP ASSIGNMENT FLOW (DORA)



8

FIREWALL MANAGEMENT



• WHAT IS A LINUX FIREWALL?

- A firewall controls which network traffic is allowed into and out of your server. In Linux, all firewall tools - iptables, ufw, and firewalld - ultimately configure the kernel's netfilter framework. The default-deny approach (block everything, then allow only what is needed) is the gold standard for production security.

• LINUX FIREWALL MANAGEMENT TOOLS

- **iptables:** iptables is a low-level firewall tool that directly interacts with the netfilter framework in the Linux kernel. It is available on almost all Linux distributions. It uses chains such as INPUT, OUTPUT, and FORWARD to define rules for packet filtering.
- **UFW (Uncomplicated Firewall):** UFW is a simplified wrapper around iptables designed to make firewall configuration easier. It is the default firewall tool on Ubuntu and Debian systems. Its configuration files are stored in the /etc/ufw/ directory.
- **firewalld:** firewalld is a dynamic firewall management daemon that uses a zone-based approach to control traffic. It is the default firewall tool on RHEL 7+, CentOS, and Fedora. It supports predefined zones such as public, internal, trusted, and drop for flexible rule management. It allows both runtime and permanent configurations.

• IPTABLES - CORE CHAINS EXPLAINED

- **INPUT Chain:** Rules for packets destined for the local machine. Example: allow SSH on port 22, block everything else. This is what protects your server from direct attacks.
- **OUTPUT chain:** Rules for packets generated by the local machine and going out. Usually permissive by default but can be restricted to prevent data exfiltration.
- **FORWARD Chain:** Rules for packets passing through the machine (used when Linux acts as a router or gateway). Kubernetes and Docker use this heavily.
- **Default Policy:** Each chain has a default action (ACCEPT or DROP) that applies when no rule matches. Always set INPUT default to DROP and then explicitly allow needed traffic.
- **Rule Order Matters:** iptables processes rules top-to-bottom and stops at the first match. The most specific rules should come first, with the catch-all DROP rule at the end.

9

SSH & REMOTE ACCESS

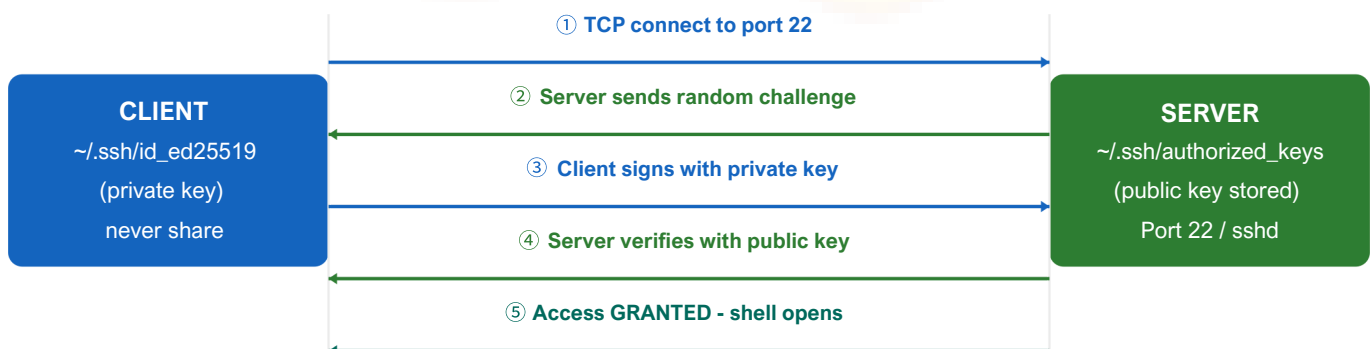


• WHAT IS SECURE SHELL (SSH)?

- SSH (Secure Shell) is the standard protocol for securely connecting to remote Linux servers over an unsecured network. It encrypts all communication - commands, outputs, and file transfers - using public-key cryptography. SSH replaced Telnet, rsh, and rlogin, which sent passwords in plain text.

- SSH provides three core capabilities: **Remote command execution** (run any command on a remote server), **Secure file transfer** (SCP and SFTP), and **Port forwarding / tunneling** (route other protocols through SSH).
- The **sshd daemon** runs on the server and listens on TCP port 22 by default. The client connects using the ssh command. The server configuration is at /etc/ssh/sshd_config.
- SSH uses **asymmetric key cryptography** : a mathematically linked key pair - a private key (kept secret on your machine) and a public key (shared with servers). Data encrypted with one key can only be decrypted by the other.
- **ED25519** keys are the modern recommended standard - faster, more secure, and smaller than RSA. Generate with: ssh-keygen -t ed25519.
- The **~/.ssh/known_hosts** file stores fingerprints of servers you have previously connected to. This prevents man-in-the-middle attacks - SSH warns you if a server fingerprint changes unexpectedly.

SSH KEY-BASED AUTHENTICATION - HOW IT WORKS



• SSH AUTHENTICATION & PORT FORWARDING

1. Password Authentication

- Password authentication uses a simple username and password to log in to a remote system.
- It is vulnerable to brute-force attacks and should be avoided in production environments.
- It is recommended to disable it by setting `PasswordAuthentication no` in the SSH configuration file.

2. Key-Based Authentication

- Key-based authentication uses a pair of cryptographic keys: a private key on the client and a public key on the server.
- The public key is stored in the `authorized_keys` file on the server, while the private key must be kept secure and never shared.
- It is the recommended method for all production SSH access due to its strong security.

3. Port Forwarding (SSH Tunneling)

- SSH port forwarding allows secure tunneling of network traffic through an encrypted SSH connection.
- Local forwarding maps a local port to a remote service, while remote forwarding exposes a local service on a remote server.
- It is widely used in DevOps to securely access internal services like databases without exposing them publicly.

• SSH CONFIGURATION FILES

File	Location	Purpose
<code>sshd_config</code>	<code>/etc/ssh/</code>	Server config - port, auth methods, allowed users
<code>ssh_config</code>	<code>/etc/ssh/</code>	System-wide client config - defaults for all users
<code>config</code>	<code>~/.ssh/</code>	Per-user client config - host aliases, key paths
<code>authorized_keys</code>	<code>~/.ssh/</code>	Public keys allowed to log into this account
<code>id_ed25519</code>	<code>~/.ssh/</code>	Private ED25519 key - permissions MUST be 600
<code>id_ed25519.pub</code>	<code>~/.ssh/</code>	Public ED25519 key - safe to share with servers
<code>id_rsa</code>	<code>~/.ssh/</code>	Private RSA key (legacy) - permissions MUST be 600
<code>known_hosts</code>	<code>~/.ssh/</code>	Server fingerprints - prevents MITM attacks

10

COMPLETE COMMANDS REFERENCE



• INTERFACE & IP ADDRESS COMMANDS

Command	Description	Example
<code>ip addr show</code>	Show all interfaces and assigned IPs	<code>ip addr show</code>
<code>ip addr show eth0</code>	Show details of a specific interface	<code>ip addr show eth0</code>
<code>ip -br addr show</code>	Brief one-line view per interface	<code>ip -br addr show</code>
<code>ip link show</code>	Show interface state (UP/DOWN) and flags	<code>ip link show</code>
<code>ip link set eth0 up</code>	Bring an interface up	<code>sudo ip link set eth0 up</code>
<code>ip addr add</code>	Temporarily assign IP to interface	<code>ip addr add 192.168.1.10/24 dev eth0</code>
<code>ip addr del</code>	Remove an IP from an interface	<code>ip addr del 192.168.1.10/24 dev eth0</code>
<code>ip -s link show</code>	Show TX/RX packet statistics per interface	<code>ip -s link show eth0</code>
<code>hostname -I</code>	Show all IP addresses on this host	<code>hostname -I</code>
<code>ifconfig</code>	Legacy interface tool (net-tools pkg)	<code>ifconfig eth0</code>
<code>ethtool eth0</code>	Show NIC physical link status and speed	<code>ethtool eth0</code>
<code>nmcli device status</code>	Show all interfaces via NetworkManager	<code>nmcli device status</code>
<code>nmcli con show</code>	List all NetworkManager connections	<code>nmcli con show</code>
<code>nmcli con up eth0</code>	Activate a network connection	<code>nmcli con up eth0</code>
<code>nmcli con down eth0</code>	Deactivate a network connection	<code>nmcli con down eth0</code>

• ROUTING COMMANDS

Command	Description	Example
<code>ip route show</code>	Display the full routing table	<code>ip route show</code>
<code>ip route show default</code>	Show only the default gateway entry	<code>ip route show default</code>
<code>ip route get 8.8.8.8</code>	Show which route a specific IP uses	<code>ip route get 8.8.8.8</code>
<code>ip route add</code>	Add a static route	<code>ip route add 10.0.0.0/8 via 192.168.1.1</code>
<code>ip route del</code>	Delete a static route	<code>ip route del 10.0.0.0/8</code>
<code>ip route add default</code>	Set the default gateway	<code>ip route add default via 192.168.1.1</code>

• ROUTING COMMANDS

Command	Description	Example
<code>ip route del default</code>	Remove the default gateway	<code>ip route del default</code>
<code>ip route replace</code>	Add or update a route atomically	<code>ip route replace 10.0.0.0/8 via 192.168.1.1</code>
<code>ip neigh show</code>	Show the ARP / neighbour table	<code>ip neigh show</code>
<code>arp -n</code>	Show ARP cache with numeric IPs (legacy)	<code>arp -n</code>
<code>route -n</code>	Display routing table (legacy net-tools)	<code>route -n</code>
<code>netstat -rn</code>	Show routing table numerically (legacy)	<code>netstat -rn</code>
<code>ipcalc 192.168.1.0/24</code>	Calculate subnet details from CIDR	<code>ipcalc 192.168.1.0/24</code>

• DNS & HOSTNAME COMMANDS

Command	Description	Example
<code>hostname</code>	Show the current hostname	<code>hostname</code>
<code>hostname -f</code>	Show the Fully Qualified Domain Name (FQDN)	<code>hostname -f</code>
<code>hostnamectl</code>	Show full hostname and OS identity info	<code>hostnamectl</code>
<code>hostnamectl set-hostname</code>	Permanently change the system hostname	<code>hostnamectl set-hostname web-01</code>
<code>dig google.com</code>	Full DNS query with detailed output	<code>dig google.com</code>
<code>dig A google.com</code>	Query only IPv4 (A) records	<code>dig A google.com</code>
<code>dig AAAA google.com</code>	Query only IPv6 (AAAA) records	<code>dig AAAA google.com</code>
<code>dig MX gmail.com</code>	Query mail exchanger records	<code>dig MX gmail.com</code>
<code>dig +short google.com</code>	Show only the resolved IP address	<code>dig +short google.com</code>
<code>dig +trace google.com</code>	Trace full DNS path from root servers	<code>dig +trace google.com</code>
<code>dig -x 8.8.8.8</code>	Reverse DNS lookup - IP to hostname	<code>dig -x 8.8.8.8</code>
<code>nslookup google.com</code>	Interactive DNS query (legacy)	<code>nslookup google.com 8.8.8.8</code>
<code>host google.com</code>	Quick DNS lookup utility	<code>host google.com</code>
<code>resolvectl status</code>	Show DNS resolver status (systemd-resolved)	<code>resolvectl status</code>
<code>resolvectl query</code>	Query DNS via systemd-resolved	<code>resolvectl query google.com</code>

• PORT, SOCKET & CONNECTIVITY COMMANDS

Command	Description	Example
<code>ss -tuln</code>	Show all listening TCP+UDP ports (no DNS lookup)	<code>ss -tuln</code>
<code>ss -tulnp</code>	Show listening ports with process name and PID	<code>ss -tulnp</code>
<code>ss -tnp</code>	Show established TCP connections with processes	<code>ss -tnp</code>
<code>ss -s</code>	Socket statistics summary by state	<code>ss -s</code>
<code>netstat -tuln</code>	Legacy: show listening ports	<code>netstat -tuln</code>
<code>netstat -anp</code>	Legacy: all connections with PIDs	<code>netstat -anp</code>
<code>lsof -i</code>	All processes with open network connections	<code>lsof -i</code>
<code>lsof -i :80</code>	Which process is using port 80	<code>lsof -i :80</code>
<code>nc -zv host port</code>	Test if a TCP port is open (netcat)	<code>nc -zv 192.168.1.1 22</code>
<code>nc -zvu host port</code>	Test if a UDP port is open	<code>nc -zvu 192.168.1.1 53</code>
<code>telnet host port</code>	Test TCP connectivity (legacy)	<code>telnet 192.168.1.1 80</code>
<code>nmap 192.168.1.1</code>	Scan all open ports on a host	<code>nmap 192.168.1.1</code>
<code>nmap -sV</code>	Scan and detect service versions	<code>nmap -sV 192.168.1.1</code>
<code>nmap -p 22,80,443</code>	Scan specific ports on a host	<code>nmap -p 22,80,443 192.168.1.1</code>
<code>curl -I</code>	Fetch HTTP headers only (HEAD request)	<code>curl -I https://google.com</code>
<code>curl -v</code>	Verbose HTTP request showing full detail	<code>curl -v http://localhost:8080</code>

• CONNECTIVITY TESTING COMMANDS

Command	Description	Example
<code>ping google.com</code>	Test ICMP reachability and round-trip time	<code>ping google.com</code>
<code>ping -c 4 8.8.8.8</code>	Send exactly 4 ping packets then stop	<code>ping -c 4 8.8.8.8</code>
<code>ping -I eth0</code>	Ping using a specific network interface	<code>ping -I eth0 192.168.1.1</code>
<code>ping6 ::1</code>	Test IPv6 connectivity to loopback	<code>ping6 ::1</code>
<code>traceroute google.com</code>	Show each hop to a destination	<code>traceroute google.com</code>
<code>traceroute -n</code>	Traceroute without DNS lookups (faster)	<code>traceroute -n 8.8.8.8</code>
<code>tracepath google.com</code>	Like traceroute but no root required	<code>tracepath google.com</code>

• CONNECTIVITY TESTING COMMANDS

Command	Description	Example
<code>mtr google.com</code>	Real-time combined ping + traceroute	<code>mtr google.com</code>
<code>mtr --report</code>	Run mtr and output a single summary report	<code>mtr --report google.com</code>
<code>wget --spider</code>	Check URL accessibility without downloading	<code>wget -q --spider http://example.com</code>

• UFW - UNCOMPLICATED FIREWALL COMMANDS

Command	Description	Example
<code>ufw enable</code>	Enable the UFW firewall	<code>sudo ufw enable</code>
<code>ufw disable</code>	Disable UFW (all rules stop)	<code>sudo ufw disable</code>
<code>ufw status verbose</code>	Show all active rules and status	<code>sudo ufw status verbose</code>
<code>ufw allow 22/tcp</code>	Allow SSH inbound	<code>sudo ufw allow 22/tcp</code>
<code>ufw allow 80/tcp</code>	Allow HTTP inbound	<code>sudo ufw allow 80/tcp</code>
<code>ufw allow 443/tcp</code>	Allow HTTPS inbound	<code>sudo ufw allow 443/tcp</code>
<code>ufw deny 23</code>	Block Telnet completely	<code>sudo ufw deny 23</code>
<code>ufw delete allow 80</code>	Remove the allow-80 rule	<code>sudo ufw delete allow 80</code>
<code>ufw allow from IP</code>	Allow all traffic from one specific IP	<code>sudo ufw allow from 10.0.0.5</code>
<code>ufw allow from IP port</code>	Allow specific IP to specific port	<code>sudo ufw allow from 10.0.0.5 to any port 22</code>
<code>ufw app list</code>	List available application profiles	<code>sudo ufw app list</code>
<code>ufw reset</code>	Reset all rules to defaults	<code>sudo ufw reset</code>
<code>ufw logging on</code>	Enable logging for blocked traffic	<code>sudo ufw logging on</code>

• FIREWALLD COMMANDS - RHEL / CENTOS / FEDORA

Command	Description	Example
<code>firewall-cmd --state</code>	Check if firewalld is running	<code>firewall-cmd --state</code>
<code>firewall-cmd --list-all</code>	Show all active rules and zone info	<code>firewall-cmd --list-all</code>
<code>firewall-cmd --get-zones</code>	List all available zones	<code>firewall-cmd --get-zones</code>
<code>--add-port=80/tcp</code>	Open port 80 (runtime - lost on reload)	<code>firewall-cmd --add-port=80/tcp</code>

• FIREWALLD COMMANDS - RHEL / CENTOS / FEDORA

Command	Description	Example
<code>--permanent --add-port=80/tcp</code>	Open port 80 permanently	<code>firewall-cmd --permanent --add-port=80/tcp</code>
<code>--add-service=http</code>	Allow HTTP service by name (runtime)	<code>firewall-cmd --add-service=http</code>
<code>--permanent --add-service=https</code>	Allow HTTPS permanently	<code>firewall-cmd --permanent --add-service=https</code>
<code>firewall-cmd --reload</code>	Reload to apply permanent rules	<code>firewall-cmd --reload</code>
<code>--remove-port=80/tcp</code>	Close port 80 (runtime)	<code>firewall-cmd --remove-port=80/tcp</code>
<code>--list-services</code>	List all currently allowed services	<code>firewall-cmd --list-services</code>

• IPTABLES COMMANDS - ALL DISTRIBUTIONS

Command	Description	Example
<code>iptables -L -n -v</code>	List all rules with counters	<code>sudo iptables -L -n -v</code>
<code>iptables -A INPUT</code>	Append rule to end of INPUT chain	<code>iptables -A INPUT -p tcp --dport 22 -j ACCEPT</code>
<code>iptables -I INPUT 1</code>	Insert rule at top of INPUT chain	<code>iptables -I INPUT 1 -p tcp --dport 80 -j ACCEPT</code>
<code>iptables -D INPUT 1</code>	Delete rule #1 from INPUT chain	<code>iptables -D INPUT 1</code>
<code>iptables -A INPUT -j DROP</code>	Drop all traffic not matched by earlier rules	<code>iptables -A INPUT -j DROP</code>
<code>iptables -F</code>	Flush all rules (clears everything)	<code>sudo iptables -F</code>
<code>iptables -P INPUT DROP</code>	Set default policy for INPUT to DROP	<code>iptables -P INPUT DROP</code>
<code>iptables-save</code>	Output all rules (for saving to file)	<code>iptables-save > /etc/iptables/rules.v4</code>
<code>iptables-restore</code>	Restore rules from saved file	<code>iptables-restore < /etc/iptables/rules.v4</code>

• SSH COMMANDS

Command	Description	Example
<code>ssh user@host</code>	Connect to remote host on default port 22	<code>ssh devuser@192.168.1.10</code>
<code>ssh -p 2222 user@host</code>	Connect on a non-standard port	<code>ssh -p 2222 devuser@192.168.1.10</code>

• SSH COMMANDS

Command	Description	Example
<code>ssh -i key.pem user@host</code>	Connect with a specific private key file	<code>ssh -i ~/.ssh/mykey.pem ec2-user@host</code>
<code>ssh-keygen -t ed25519</code>	Generate a new ED25519 key pair (recommended)	<code>ssh-keygen -t ed25519 -C "me@example.com"</code>
<code>ssh-keygen -t rsa -b 4096</code>	Generate a 4096-bit RSA key pair (legacy)	<code>ssh-keygen -t rsa -b 4096</code>
<code>ssh-copy-id user@host</code>	Copy public key to remote authorized_keys	<code>ssh-copy-id devuser@192.168.1.10</code>
<code>ssh-add ~/.ssh/id_ed25519</code>	Add key to SSH agent for passwordless session	<code>ssh-add ~/.ssh/id_ed25519</code>
<code>scp file user@host:/path</code>	Securely copy a file to a remote host	<code>scp app.tar.gz user@host:/opt/</code>
<code>scp -r dir user@host:/path</code>	Recursively copy a directory to remote	<code>scp -r ./project user@host:/opt/</code>
<code>scp user@host:/file ./local</code>	Copy a file from remote to local	<code>scp user@host:/var/log/app.log ./</code>
<code>sftp user@host</code>	Start interactive secure file transfer session	<code>sftp devuser@192.168.1.10</code>
<code>ssh -L 8080:localhost:80</code>	Local port forward - local:8080 → remote:80	<code>ssh -L 8080:localhost:80 user@host</code>
<code>ssh -R 9090:localhost:3000</code>	Remote port forward — remote:9090 → local:3000	<code>ssh -R 9090:localhost:3000 user@host</code>
<code>ssh -D 1080 user@host</code>	Dynamic SOCKS proxy through SSH tunnel	<code>ssh -D 1080 user@host</code>
<code>systemctl status sshd</code>	Check if SSH server daemon is running	<code>systemctl status sshd</code>
<code>systemctl restart sshd</code>	Restart SSH after config changes	<code>sudo systemctl restart sshd</code>

• TRAFFIC CAPTURE & ANALYSIS

Command	Description	Example
<code>tcpdump -i eth0</code>	Capture all traffic on eth0	<code>sudo tcpdump -i eth0</code>
<code>tcpdump -n port 80</code>	Capture only HTTP port 80 traffic	<code>sudo tcpdump -n port 80</code>
<code>tcpdump host 10.0.0.1</code>	Capture traffic to/from one specific IP	<code>sudo tcpdump host 10.0.0.1</code>
<code>tcpdump -w file.pcap</code>	Write captured packets to file for later analysis	<code>sudo tcpdump -w capture.pcap</code>
<code>tcpdump -r file.pcap</code>	Read and display a previously saved capture	<code>tcpdump -r capture.pcap</code>

• TRAFFIC CAPTURE & ANALYSIS

Command	Description	Example
<code>iftop -i eth0</code>	Real-time bandwidth usage per connection	<code>sudo iftop -i eth0</code>
<code>nethogs eth0</code>	Network bandwidth usage broken down by process	<code>sudo nethogs eth0</code>
<code>nload eth0</code>	Simple real-time network throughput graph	<code>nload eth0</code>
<code>sar -n DEV 1 5</code>	Network stats every 1 second for 5 intervals	<code>sar -n DEV 1 5</code>
<code>iperf3 -s</code>	Start iperf3 server for bandwidth testing	<code>iperf3 -s</code>
<code>iperf3 -c host</code>	Run bandwidth test against an iperf3 server	<code>iperf3 -c 192.168.1.1</code>
<code>vnstat -i eth0</code>	Historical daily/monthly traffic statistics	<code>vnstat -i eth0</code>
<code>netstat -i</code>	Interface statistics table (legacy net-tools)	<code>netstat -i</code>

• NETWORK SERVICES MANAGEMENT COMMANDS

Command	Description	Example
<code>systemctl status NetworkManager</code>	Check NetworkManager service status	<code>systemctl status NetworkManager</code>
<code>systemctl restart NetworkManager</code>	Restart NetworkManager	<code>sudo systemctl restart NetworkManager</code>
<code>nmcli con add</code>	Create a new connection profile	<code>nmcli con add type ethernet ifname eth0</code>
<code>systemctl status systemd-networkd</code>	Check systemd-networkd status	<code>systemctl status systemd-networkd</code>
<code>resolvectl status</code>	Check systemd-resolved DNS status and servers	<code>resolvectl status</code>
<code>dhclient eth0</code>	Manually request a DHCP lease on interface	<code>sudo dhclient eth0</code>
<code>dhclient -r eth0</code>	Release current DHCP lease	<code>sudo dhclient -r eth0</code>
<code>netplan apply</code>	Apply Netplan network config (Ubuntu)	<code>sudo netplan apply</code>
<code>netplan try</code>	Test Netplan config with auto-rollback	<code>sudo netplan try</code>
<code>cat /proc/net/dev</code>	Raw kernel-level interface statistics	<code>cat /proc/net/dev</code>